Programmer Manual

מערכת לאיתור מושבים פנויים ברכבת

קבוצה ב2 · אוקטובר 2026

המסמך הזה מיועד למתכנת שממשיך את הפרויקט אחרינו. הוא מניח ידע בסיסי בתכנות ולא מניח היכרות עם ארדואינו או עם הפרויקט.

סקירה

המערכת מזהה אם מושב ברכבת תפוס, ומציגה לנוסע על הרציף לאיזה קרון כדאי לעלות. מעל כל מושב יש חיישן מרחק על-קולי; בקר אחד קורא את כולם, מחליט תפוס או פנוי, ומוציא שתי תוצאות: צבע על רצועת לדים, ודף אינטרנט שהוא מגיש בעצמו.

הכל רץ על בקר אחד. אין שרת חיצוני, אין מסד נתונים ואין ענן.

מה יש במאגר

נתיב	מה זה
`ESP32/seat_guide/seat_guide.ino`	התוכנית הראשית. כל מה שרץ על הבקר נמצא שם
`Unit Tests/Sensor_Check/Sensor_Check.ino`	סקיץ אבחון לחיישנים בלבד, בלי רשת ובלי לוגיקת החלטה
`Documentation/`	גליון המידות של הדגם ותיעוד החיווט

הערת אבטחה. שם הרשת והסיסמה בעותק שבמאגר הם מילות ממלא מקום בלבד. צריך להחליף אותם מקומית לפני העלאה לבקר, ולא להעלות את הסיסמה האמיתית חזרה למאגר.

חומרה וחיווט

רכיב	כמות	תפקיד
ESP32 DevKit V1	1	הבקר. קורא חיישנים, מחליט, מרים שרת
HC-SR04	2	חיישן מרחק על-קולי, אחד לכל מושב
WS2812B	רצועה של 3	נורית לכל קרון

מטריית ניסוי	1	מחלקת מתח והארקה לשני החיישנים

טבלת הפינים

פין	מחובר אל	כיוון
27	TRIG של מושב A	פלט
26	ECHO של מושב A	קלט
13	TRIG של מושב B	פלט
35	ECHO של מושב B	קלט בלבד
14	נתונים של רצועת הלדים	פלט
5V ו-GND	הזנת שני החיישנים והרצועה	—

פין 35 הוא לא שרירותי. בבקר הזה הוא פין קלט בלבד — החומרה לא מאפשרת לכתוב עליו. זה הופך אותו לבחירה בטוחה לקו ההד, שבוודאי אף פעם לא צריך להיכתב. אם מעבירים את הפרויקט לבקר אחר, כדאי לבדוק איזה פינים שם הם קלט בלבד.

החיישנים צורכים 5 וולט. הם לא עובדים אמין על 3.3 וולט. המתח מגיע מפין ה-5V של הבקר דרך המטרייה.

סביבת פיתוח

מה להתקין

1. **Arduino IDE**, גרסה 2 ומעלה.
2. **תמיכת ESP32.** בהעדפות מוסיפים את כתובת מנהל הלוחות של Espressif, ואז מתקינים את חבילת esp32 ממנהל הלוחות.
3. **הספרייה Adafruit NeoPixel**, ממנהל הספריות.

שתי הספריות האחרות שהתוכנית משתמשת בהן, `WiFi` ו-`WebServer`, מגיעות עם תמיכת ה-ESP32 ואין צורך להתקין אותן בנפרד.

הגדרות ההעלאה

הגדרה	ערך
Board	DOIT ESP32 DEVKIT V1
Upload Speed	921600
Port	הפורט שמופיע כשמחברים את הבקר

Serial Monitor	115200

לפני ההעלאה הראשונה

בראש הקובץ יש שתי שורות שחייבים להתאים לרשת שלכם:

```
const char* WIFI_NAME     = "YOUR_HOTSPOT_NAME";
const char* WIFI_PASSWORD = "YOUR_HOTSPOT_PASSWORD";
```

הרשת חייבת להיות 2.4 ג'יגההרץ. הבקר לא ייראה רשת של 5 ג'יגההרץ גם אם הסיסמה נכונה, וזו הסיבה הנפוצה ביותר ל"הכול נראה תקין אבל אין חיבור".

בדיקה שהכל עובד

אחרי העלאה, פותחים את המסך הסדרי. אם הכל תקין מופיעים שלושה דברים בסדר הזה: שורות פתיחה עם הספים של כל מושב, שורה שמתחילה ב-`PASS` עם כתובת ה-IP, ואז שורת מצב חוזרת לכל מושב עם המדידה הגולמית, המוחלקת והמצב.

אם רוצים לבדוק רק את החיישנים, בלי רשת ובלי לוגיקה, מעלים את `Sensor_Check.ino` במקום. הוא מדפיס ממוצע, מינימום ומקסימום על חלון של עשרים מדידות, וזה הכלי הנכון לכיול ולאיתור חיווט רעוע.

הקבועים

כל ההתנהגות של המערכת נשלטת מעשר קבועים בראש הקובץ. אין צורך לגעת בשום דבר אחר כדי לכוון את המערכת.

קבוע	ערך	על מה הוא שולט	מתי לשנות
`TAKEN_BELOW`	מערך, 8.5 לשניהם	מתחת למרחק הזה המושב תפוס	בכל התקנה חדשה
`FREE_ABOVE`	מערך, 10.5 לשניהם	מעל המרחק הזה המושב פנוי	בכל התקנה חדשה
`PINGS`	5	כמה פעימות בכל מדידה	כמעט לעולם לא
`MIN_VALID_CM`	2.5	מתחת לזה הקריאה נזרקת	אם החיישן קרוב מאוד למושב
`MAX_VALID_CM`	25.0	מעל זה הקריאה נזרקת	בקרון גבוה יותר
`STAY_TIME`	2000	כמה זמן שינוי חייב להחזיק	אם המערכת איטית או רועדת מדי
`ERROR_TIME`	3000	כמה זמן בלי הד עד מצב תקלה	כמעט לעולם לא

`SMOOTHING`	0.5	כמה מהערך הקודם נשמר	אם הקריאה רועדת אחרי החציון
`CAR_SEATS`	2, 10, 10	כמה מושבים בכל קרון	בהרחבה
`CAR_COUNT`	3	כמה קרונות ברכבת	בהרחבה

השניים הראשונים הם מערכים, וזה לא מקרי. שני חיישנים באותה תקרה נותנים מדידות ריקות שונות זו מזו, וסף משותף גורם לאחד מהם להיתקע. פירוט בפרק המלכודות.

איך מחשבים את שני הראשונים. מודדים מושב ריק ומושב תפוס, ומחשבים:

```
G = empty - occupied
TAKEN_BELOW = (empty + occupied) / 2  -  G / 7
FREE_ABOVE  = (empty + occupied) / 2  +  G / 7
```

הכלל שחייב להתקיים: המדידה הריקה חייבת להיות גדולה מ-`FREE_ABOVE`. אחרת המושב ייתקע על תפוס ולא יחזור.

מבנה התוכנית

הקובץ הוא כ-688 שורות, אבל כ-380 מהן הן דף התצוגה השמור כמחרוזות. לוגיקת החיישנים היא כ-300 שורות.

אזור	מה יש שם
ספריות והגדרות	פרטי הרשת, הפינים, הקבועים
משתני המצב	חמישה מערכים, אחד לכל מושב
דף התצוגה	שלוש מחרוזות `PROGMEM` — עמוד, עיצוב, סקריפט
פונקציות החיישן	פעימה אחת, חציון, החלטה
פונקציות הקרון	תרגום מושב לקוד, ספירה, בחירת הקרון הטוב
הנורית	תרגום מספר פנויים לצבע
השרת	חמש כתובות וחיבור מחדש לרשת
הפעלה ולולאה	מה רץ פעם אחת ומה רץ תמיד

משתני המצב

```
SeatState     seatState[2];    // FREE_SEAT / TAKEN_SEAT / FAULT_SEAT
float         smoothDist[2];   // the smoothed reading
float         lastRaw[2];      // the last raw reading, for the monitor
unsigned long changeStart[2];  // when the pending change began
unsigned long lastEcho[2];     // when an echo last came back
```

כל חמישתם מערכים באורך מספר המושבים. זו הסיבה שהרחבה לעשרה מושבים היא שינוי מספרים ולא כתיבה מחדש.

הלולאה הראשית

```
void loop() {
  updateSeat(0);
  delay(100);
  updateSeat(1);
  delay(100);
  updateLights();
  keepWifi();
  server.handleClient();
}
```

זה כל הזרימה. כל סיבוב לוקח כ-270 אלפיות השנייה, כלומר כארבע מדידות בשנייה לכל מושב. ההשהיות הן לא מקריות — פירוט בפרק המלכודות.

האלגוריתם

שלוש פונקציות, אחת על גבי השנייה.

1. פעימה אחת

```
digitalWrite(trigPin, HIGH);
delayMicroseconds(10);
digitalWrite(trigPin, LOW);
long duration = pulseIn(echoPin, HIGH, 30000);
if (duration == 0) return 0;
return duration * 0.0343 / 2.0;
```

פולס של עשר מיליוניות השנייה מפעיל את החיישן. `pulseIn` מודד כמה זמן קו ההד נשאר גבוה. מכפילים במהירות הקול ומחלקים בשתיים, כי הקול עשה את הדרך פעמיים.

הערך `30000` הוא פסק זמן. בלעדיו הבקר נתקע לנצח כשחיישן מתנתק.

2. חציון של חמש

```
for (int k = 0; k < PINGS; k++) {
  float d = onePing(trigPin, echoPin);
  if (d >= MIN_VALID_CM && d <= MAX_VALID_CM) v[n++] = d;
  delay(6);
}
// insertion sort over v[0..n)
return v[n / 2];
```

הערך האמצעי, לא הממוצע. הרעש של חיישן על-קולי הוא לא רעש קטן סביב האמת — הוא הד חריג בודד ממשטח אחר, שנופל רחוק מהאמת. ממוצע נסחב אחריו; חציון מתעלם ממנו לגמרי.

3. ההחלטה

סדר הפעולות חשוב:

1. אם לא חזר הד בכלל — בודקים כמה זמן עבר מאז ההד האחרון, ואולי עוברים למצב תקלה. **מחזירים מיד** — אין החלטה בלי מידע.
2. מחליקים את הערך.
3. אם החיישן התאושש מתקלה — קובעים מצב מיד, בלי המתנה.
4. רק אז שני הספים וההמתנה.

```
bool wantsChange =
  (seatState[i] == FREE_SEAT  && smoothDist[i] < TAKEN_BELOW[i]) ||
  (seatState[i] == TAKEN_SEAT && smoothDist[i] > FREE_ABOVE[i]);

if (!wantsChange) { changeStart[i] = 0; return; }
if (changeStart[i] == 0) changeStart[i] = now;
if (now - changeStart[i] >= STAY_TIME) { /* flip the state */ }
```

שני התנאים הם ההיסטרזיס: מצב פנוי נבדק רק מול הסף התחתון, מצב תפוס רק מול העליון. בין השניים אף תנאי לא מתקיים והמצב נשמר.

`changeStart` מתאפס בכל סיבוב שבו התנאי לא מתקיים. לכן מעבר חולף לא מצטבר לשתי שניות.

הממשק

הבקר מרים שרת על פורט 80 ועונה לחמש כתובות.

כתובת	מחזירה
`/`	עמוד התצוגה
`/s.css`	גיליון העיצוב

`/a.js`	הסקריפט
`/data`	מצב כל המושבים, כ-JSON
`/toggle`	הופך מושב מדומה

מבנה הנתונים

כל מושב מיוצג כמספר אחד:

קוד	משמעות
0	פנוי
1	תפוס
2	תקלת חיישן

התשובה של `/data`:

```
{
  "best": 1,
  "cars": [
    { "seats": [0, 1] },
    { "seats": [0,0,1,0,0,1,0,0,0,0] },
    { "seats": [1,1,1,0,1,1,1,1,0,1] }
  ]
}
```

`best` הוא אינדקס הקרון עם הכי הרבה מושבים פנויים, או מינוס אחת אם הרכבת מלאה. קרון 0 הוא האמיתי; השאר מדומים.

`/toggle` מקבלת שני פרמטרים, `car` ו-`seat`, ומתעלמת מבקשות על קרון 0 — אי אפשר לזייף מושב אמיתי מהדפדפן.

מחזור העדכון

הדף פונה אל `/data` פעם בשנייה. אם הבקשה נכשלת, הדף עובר למצב הדגמה עם נתוני דוגמה ומסמן זאת במחוון החיבור. זו הסיבה שהדף נראה תקין גם כשהוא לא מחובר לכלום.

איך מרחיבים

עוד מושבים בקרון האמיתי

זה המקרה הנפוץ. ארבעה שלבים:

1. להאריך את מערכי הפינים ואת מערכי הספים, עם ערך לכל מושב חדש.

2. להאריך את חמשת מערכי המצב באותו אורך.
3. לעדכן את הערך הראשון ב-`CAR_SEATS`.
4. להוסיף קריאה ל-`updateSeat` עם האינדקס החדש בלולאה, עם `delay(100)` אחריה.

שום לוגיקה לא משתנה. כל הפונקציות כבר עובדות על אינדקס מושב.

> **מגבלה.** כל מושב צורך שני פינים, ולבקר יש מספר מוגבל של פינים פנויים. מעבר לכעשרה מושבים צריך מרחיב פינים או בקר שני. ויש גם זמן: כל מושב מוסיף כ-135 אלפיות השנייה לכל סיבוב, אז עשרה מושבים יורידו את קצב המדידה לכמדידה בשנייה למושב. זה עדיין סביר מול המתנה של שתי שניות, אבל כדאי לדעת.

עוד קרונות

מעדכנים את `CAR_COUNT` ומוסיפים ערך ל-`CAR_SEATS`. צריך גם לוודא שברצועת הלדים יש מספיק נוריות — הרצועה מאותחלת באורך `CAR_COUNT`.

בצד הדפדפן, המערך `CARS` בראש הסקריפט צריך להתאים באורכו למה ש-`/data` מחזירה. זה המקום היחיד שבו השרת והדף חייבים להסכים זה עם זה.

להעביר לחומרה אחרת

התוכנית תלויה ב-ESP32 בשלושה דברים בלבד: ספריית הרשת, ספריית השרת, וההנחה שפין 35 הוא קלט בלבד. כל השאר — החיישנים, החציון, ההיסטרזיס — הוא ארדואינו רגיל.

מלכודות

כל אחת מהן עלתה לנו זמן. כדאי לקרוא את הפרק הזה לפני שמשנים משהו.

סף משותף לשני מושבים — הבאג הגדול ביותר

בגרסה הראשונה הספים היו מספר בודד, משותף לשני המושבים. מושב A עבד מצוין; מושב B נתקע על תפוס ולא חזר לפנוי לעולם.

הסיבה: הקריאה הריקה של B הייתה נמוכה מזו של A, ונפלה בתוך האזור האפור בין שני הספים. מושב יכול לרדת מתחת לסף התחתון ולהיהפך לתפוס, אבל כדי לחזור הוא צריך לעלות מעל הסף העליון — והקריאה הריקה שלו פשוט לא הגיעה לשם.

> זה נראה בדיוק כמו חיישן מקולקל, וזו הסיבה ששווה להכיר אותו. **הכלל: לכל מושב הספים שלו.**

ממוצע במקום חציון

הגרסה הראשונה לקחה ממוצע של כמה פעימות. מושב B היה מגיב באיטיות ולפעמים לא בכלל, כי קריאות חריגות מקצה הדופן משכו את הממוצע והקפיצו אותו הלוך וחזור מעל הסף, וכל קפיצה איפסה את מונה השתי שניות. המעבר לחציון פתר את זה לגמרי.

ההשהיות בלולאה הן לא מילוי מקום

`delay(100)` אחרי כל מושב נותן להד של החיישן הקודם לדעוך לפני שהבא יורה. שני חיישנים על-קוליים משתמשים באותה תדירות, וחיישן אחד שמקבל את ההד של השני מדווח מרחק שגוי. אם מורידים את ההשהייה הזאת המערכת תיראה מהירה יותר ותהיה פחות נכונה.

גם `delay(6)` בין הפעימות הוא מאותו טעם.

פין 35 לא יכול להיות פלט

אם מחליפים פינים ומעבירים בטעות את `TRIG` לפין 35, החיישן פשוט לא יירה ולא תהיה שום הודעת שגיאה. המערכת תראה כאילו החיישן מת.

דף התצוגה מפוצל לשלוש בקשות

כשהכל היה בקשה אחת גדולה, הבקר לא הצליח לשלוח אותה בשלמות — אין לו מספיק זיכרון. הפיצול לעמוד, עיצוב וסקריפט הוא לא סגנון — הוא הכרחי. אל תאחדו אותם חזרה.

וכל שלושתן חייבות להישאר `PROGMEM`. בלעדיו הטקסט נטען לזיכרון העבודה ולא נשאר מספיק עבור הרשת.

הסיסמה בקובץ

שם הרשת והסיסמה כתובים בתוך הקוד. זה בסדר לפרויקט לימודי, אבל לא למערכת אמיתית. העותק שבמאגר מכיל מילות ממלא מקום בלבד, וכך זה צריך להישאר.